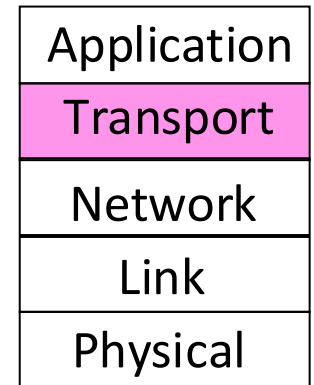


Control de congestión

Cómo evitar que la red colapse cuando todos quieren enviar a la vez

CAPA DE TRANSPORTE



¿Qué es la congestión?

DEFINICIÓN

Cuando la red no puede seguir el ritmo del tráfico que le llega

Si un emisor manda más datos de los que la subred puede transportar, los **enrutadores intermedios** se saturan, los búferes se llenan y empiezan a **descartar paquetes**.

CÓMO OCURRE

01

Llegan demasiados paquetes

El tráfico entrante supera la capacidad de reenvío del enrutador.

02

Los búferes se desbordan

La cola de salida se llena y el enrutador no puede almacenar más.

03

Se descartan paquetes

El enrutador tira segmentos; el emisor deberá retransmitirlos.

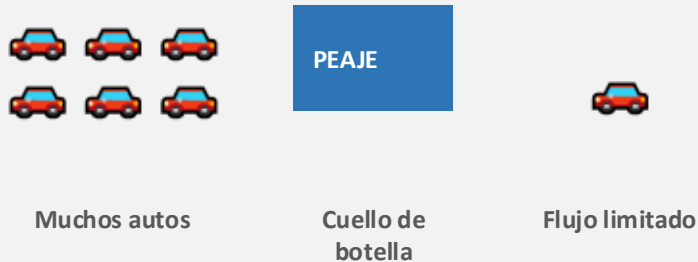
Sin un mecanismo de control, la situación se retroalimenta: más retransmisiones generan aún más tráfico.

El problema a resolver

PROBLEMA

Necesitamos controlar el ritmo de envío según la capacidad real de la red

ANALOGÍA: UNA AUTOPISTA CON PEAJE



Si llegan más autos de los que pasan, se forma cola. Si nadie los modera, colapsa.

EN LA RED REAL

- Los **autos** son los **paquetes** que viajan.
- El **peaje** es el **enrutador** más lento del camino.
- Si los búferes del enrutador se llenan, **descarta paquetes**.
- Necesitamos que el **emisor** regule su ritmo según la capacidad real.

La capa de transporte (TCP en el emisor) es quien regula el ritmo — los enrutadores solo reenvían o descartan.

¿Cómo controla la congestión TCP?

MECANISMO GENERAL

TCP mide, detecta y reacciona — ajustando cuántos datos tiene "en vuelo"

01

MEDIR

TCP mantiene la **ventana de congestión (VC)**: el máximo de bytes que el emisor puede tener sin confirmar en un momento dado.

02

DETECTAR

El emisor reconoce la congestión cuando **se pierden paquetes** — observable por **timeouts** o **ACKs duplicados**.

03

REACCIONAR

Cuando detecta congestión, el emisor **reduce** el tamaño de la VC para enviar menos datos y aliviar la red.

Ventana de congestión (VC) = bytes que el emisor puede tener en la red sin haber recibido ACK todavía.

*Ventana de congestión (VC) = **congestion window** o **cwnd***

¿Cómo detecta TCP la congestión?

DETECCIÓN

TCP infiere la congestión a partir de la pérdida de paquetes

Cuando expira el temporizador de retransmisión, el paquete probablemente se perdió. La pregunta clave: ¿por qué?

DOS POSIBLES CAUSAS DE UNA PÉRDIDA

CAUSA A

Ruido en el canal (capa física/enlace)

Un error físico corrompe los bits y el paquete se descarta.

→ *Rara hoy con fibra óptica*

CAUSA B

Enrutador congestionado

Los búferes están llenos y el enrutador descarta paquetes.

→ *Causa dominante en Internet*

Por eso todos los algoritmos de congestión de TCP asumen que una pérdida = congestión.

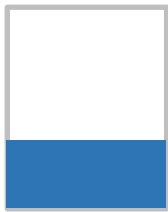
La siguiente pregunta: ¿qué tamaño le damos a la VC?

EL DILEMA

Si la VC es muy chica desperdiciamos capacidad; si es muy grande generamos congestión

CÓMO SE VE EN LA PRÁCTICA

VC MUY PEQUEÑA



SUBUTILIZACIÓN

El emisor manda pocos bytes por RTT.
Sobra capacidad en la red.

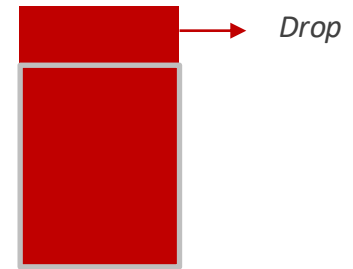
VC JUSTA · ÓPTIMO



USO PLENO

El emisor usa toda la capacidad disponible sin saturar los búferes.

VC DEMASIADO GRANDE



CONGESTIÓN

Los búferes se saturan, se descartan paquetes y aumentan las retransmisiones.

Idea: empezar con poco y aumentar hasta que la red indique que llegamos al límite → arranque lento.

Arranque lento (Slow Start)

IDEA CENTRAL

Empezar con una ventana pequeña y duplicarla hasta detectar que la red se satura

LA PROGRESIÓN · UNO, DOS, CUATRO, OCHO...



Cada RTT duplica los segmentos en vuelo — **crecimiento exponencial** hasta encontrar el límite.

Se llama “*slow*” porque **arranca desde 1 segmento** en lugar de saturar la red de entrada — aunque la ventana luego crezca exponencial, **el punto de partida es mínimo**.

“The throughput from LBL to UC Berkeley dropped from 32 Kbps to 40 bps.” — V. Jacobson, SIGCOMM '88 (quien inventó el slow start)

Arranque lento: paso a paso

EJECUCIÓN DEL ALGORITMO

01 Inicio

Emisor y receptor acuerdan el **Segmento de Tamaño Máximo (STM)**. La VC arranca en 1 STM y se envía 1 segmento.

02 Confirmación

Si el ACK llega antes del timeout, la VC crece en 1 STM por cada ACK recibido en esa ráfaga.

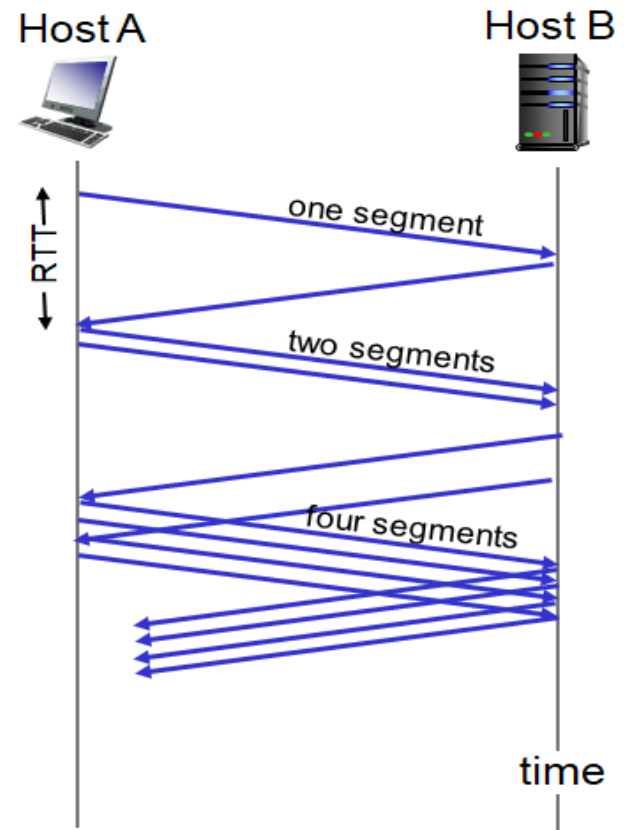
03 Duplicación por RTT

Al confirmar n segmentos, la VC pasa de n a $2n \rightarrow$ el ritmo crece exponencialmente.

04 Hasta el límite

Crece hasta que ocurre un timeout o se alcanza el tamaño de la ventana del receptor.

EN EL DIAGRAMA $\rightarrow$



Cada RTT duplica la cantidad de segmentos en vuelo: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \dots$

Crecimiento exponencial y reacción al timeout

LAS DOS REGLAS CLAVE

SI ACK A TIEMPO

$VC \leftarrow VC + n \text{ segmentos}$

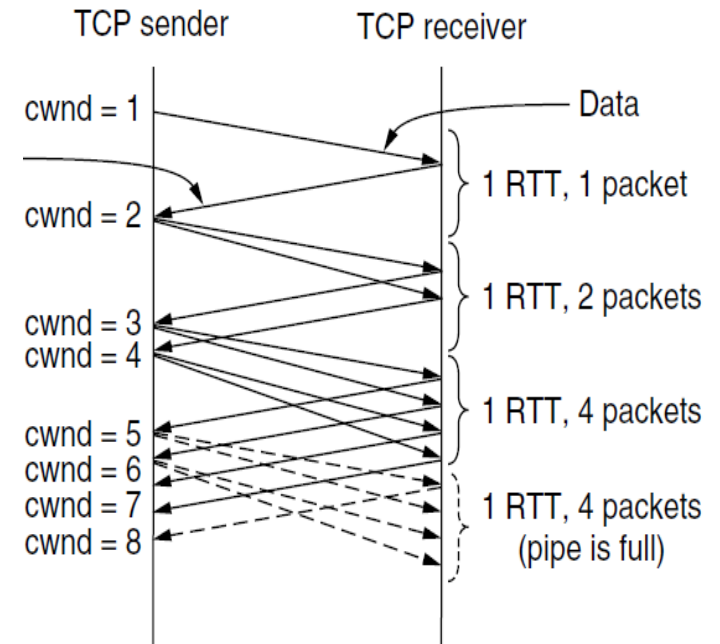
Cuando la VC es de **n** segmentos y los **n** ACK llegan a tiempo, se agregan **n** STM a la VC. Resultado: *duplicación* por RTT.

SI HAY TIMEOUT

$VC \leftarrow VC / 2$

La VC se recorta a la mitad y no se enviarán ráfagas mayores a ese nuevo valor. Es la primera forma de **reaccionar a la congestión**.

EN EL DIAGRAMA →



Versión previa a Tahoe: el algoritmo se mejorará más adelante.

Ejercicio: tiempo hasta llenar la ventana

EJERCICIO · ARRANQUE LENTO

¿Cuánto tarda arranque lento en enviar una ventana completa?

DATOS

- RTT: **10 ms**
- Ventana del receptor: **24 KB**
- Tamaño máximo de segmento (STM): **2 KB**
- Condición: **sin congestión en la red**

CONSIGNA

¿Cuánto tiempo pasa desde el primer segmento enviado hasta mandar una ventana completa de **24 KB**?

PISTA — PROGRESIÓN DE LA VC

2 → 4 → 8 → 16 → 24 KB

Contar los RTT y multiplicar por 10 ms.

RESPUESTA

Hacen falta **4 RTT** para pasar de 2 KB a 24 KB → **40 ms**.

Pista: la ventana se duplica cada RTT — 2 → 4 → 8 → 16 → 24 KB.

Dos problemas del arranque lento

CRÍTICAS AL ALGORITMO ORIGINAL

Reaccionar solo con "timeout $\rightarrow$ VC / 2" es demasiado lento y demasiado agresivo

CRÍTICA 1 · REACCIÓN EXCESIVA

Cortar a la mitad puede ser demasiado

PROBLEMA

Un único timeout no siempre indica saturación severa. La red podía soportar más que la mitad de la VC actual.

CONSECUENCIA

Se desaprovecha capacidad disponible de la subred.

CRÍTICA 2 · DETECCIÓN LENTA

El timeout tarda mucho en dispararse

PROBLEMA

Hasta que expire el temporizador, el emisor no sabe que hubo pérdida y **sigue inyectando paquetes con una VC mayor a lo que la red soporta.**

CONSECUENCIA

Se agrava la congestión y la retransmisión llega tarde.

Próximo paso: un mecanismo para reconocer pérdidas antes del timeout

¿Cómo detectar pérdida antes del timeout?

ATAcando LA CRÍTICA 2

Observación clave: los ACK que llegan al emisor traen información útil aun cuando no confirman nuevos datos

PREMISA

- Asumimos que **cada segmento que llega al receptor dispara un ACK**.
- El ACK siempre indica el **próximo byte esperado** — es acumulativo.
- Si se pierde un segmento pero llegan los posteriores, el receptor envía **ACKs duplicados** que confirman el último byte en orden recibido.
- Esto significa que el emisor puede **"oír" la pérdida vía los ACKs duplicados** sin esperar a que expire el temporizador.

Si aprovechamos esa señal podemos reaccionar mucho más rápido que esperando un temporizador.

¿Qué es un ACK duplicado?

ESCENARIO TÍPICO

Si se pierde el segmento N pero llegan N+1, N+2, N+3 ... el receptor insiste: "todavía espero N"

- El receptor confirma el **último byte recibido en orden**.
- Al perderse un segmento y llegar los posteriores, todos los ACK que envía dicen **lo mismo**.
- A cada uno de esos ACK repetidos se les llama **ACK duplicado**.
- Llegan mucho antes que expire el temporizador del emisor.

SECUENCIA DE ACKs

Segmento N → **X (se pierde)** —

Segmento N+1 → **ACK "espero N"** *1er duplicado*

Segmento N+2 → **ACK "espero N"** *2do duplicado*

Segmento N+3 → **ACK "espero N"** *3er duplicado*

Un único ACK duplicado aislado puede no significar pérdida (ver siguiente slide).

Pero cuidado: no todo ACK duplicado implica pérdida

INTERPRETACIÓN

Los paquetes también pueden llegar desordenados y disparar ACKs duplicados espurios

CASO A · REORDENAMIENTO

Falso positivo

Los segmentos pueden tomar rutas distintas y llegar fuera de orden al receptor. Se disparan 1–2 ACKs duplicados pero ningún paquete se perdió.

→ *Pocos duplicados*

CASO B · PÉRDIDA REAL

Pérdida verdadera

El segmento N no llega al receptor. Los segmentos posteriores sí, y cada uno dispara un ACK repitiendo "espero N".

→ *Muchos duplicados seguidos*

¿Cómo distinguir los dos casos? Necesitamos un umbral confiable.

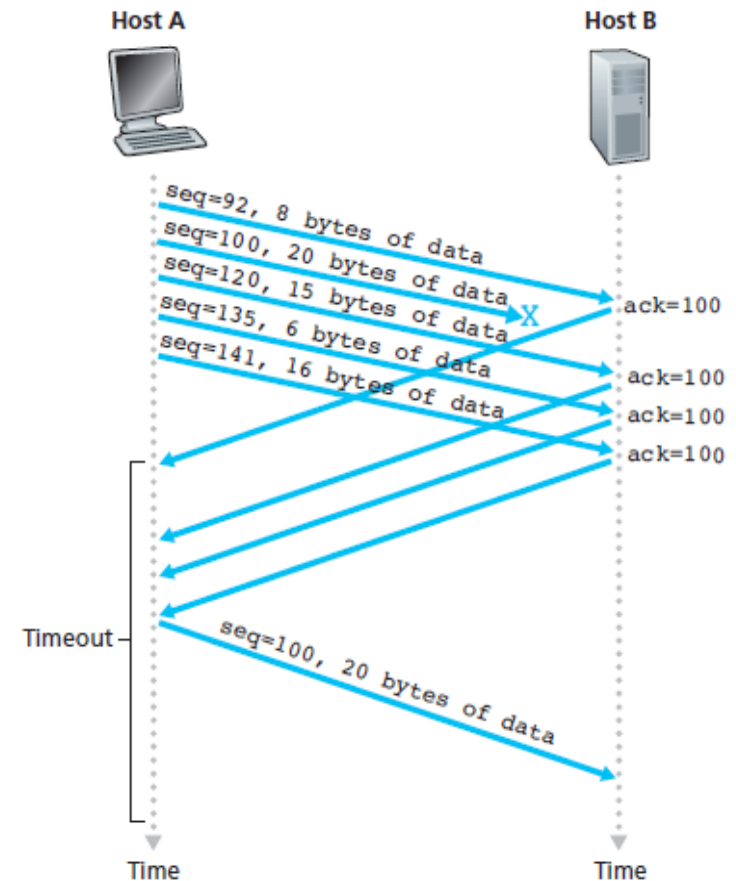
Retransmisión rápida: la regla de los 3 ACKs

HEURÍSTICA

3 ACKs duplicados → asumimos que el segmento se perdió y lo retransmitimos ya

- 01** El emisor cuenta ACKs duplicados consecutivos.
- 02** Al recibir el tercero asume que el segmento se perdió.
- 03** Lo retransmite inmediatamente, sin esperar al temporizador.
- 04** Así evita la latencia del timeout y se recupera rápido.

EN EL DIAGRAMA →



El emisor ve 3 ACKs repetidos y retransmite sin esperar el timeout.

TCP Tahoe: la primera solución completa

ALGORITMO DE CONTROL DE CONGESTIÓN DE INTERNET

Combina arranque lento con un umbral que divide las fases de crecimiento

IDEA NUEVA: EL UMBRAL — SSTHRESH (slow-start threshold · se recalcula en cada pérdida)

DEBAJO DEL UMBRAL

Arranque lento

La VC se duplica cada RTT — crecimiento exponencial — para encontrar rápido la capacidad de la red.

SOBRE EL UMBRAL

Evitación de congestión

La VC crece de manera lineal (+1 **MSS** por RTT). Nos acercamos con cuidado a la capacidad máxima.

MSS = Maximum Segment Size (= STM en castellano).

Ante pérdida — timeout o 3 ACKs duplicados, Tahoe reacciona igual: ① $ssthresh \leftarrow VC \text{ actual} / 2$ ② $VC \leftarrow 1 \text{ MSS}$

El umbral no es fijo: se actualiza en cada evento de pérdida con la VC del momento.

Reacción ante pérdida: ciclo completo de Tahoe

QUÉ PASA CUANDO SE DETECTA CONGESTIÓN

Tahoe reinicia con arranque lento pero recuerda dónde estaba la red antes del corte

1 Se detecta pérdida *(timeout o 3 ACKs duplicados)*

El emisor registra el evento de congestión.

2 Se guarda la mitad como umbral *$ssthresh \leftarrow VC / 2$*

Es la "capacidad estimada segura" de la red.

3 La ventana vuelve al mínimo *$VC \leftarrow 1 MSS$*

Se empieza desde cero porque **no sabemos** si el estado de la red cambió.

4 Arranque lento hasta el umbral, luego crecimiento lineal *$exp \rightarrow lineal en ssthresh$*

Debajo del umbral se duplica; arriba solo +1 MSS por RTT.

El umbral (ssthresh) es la memoria del algoritmo: "hasta aquí funciona, acá hubo problema".

Estado estable: el "diente de sierra"

COMPORTAMIENTO A LARGO PLAZO

Tahoe busca continuamente el límite de la red — sube, pierde, baja, repite

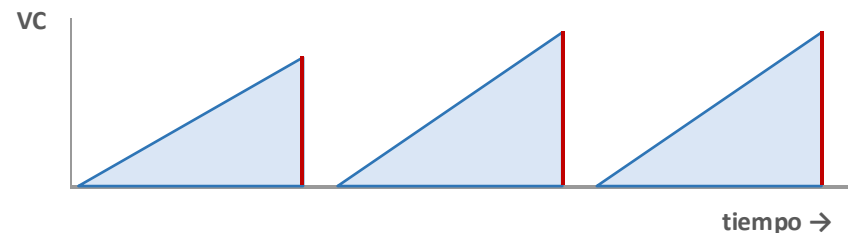
PATRÓN TÍPICO

- La VC sube de forma lineal en evitación de congestión.
- Cuando llega al punto de saturación, se pierde un paquete.
- La VC se corta a la mitad (pasa a ser el nuevo umbral) y vuelve a 1 MSS.
- Arranque lento rápido hasta el umbral, luego crecimiento lineal otra vez.
- **Este ciclo es la forma clásica "diente de sierra" AIMD.**

AIMD = Additive Increase, Multiplicative Decrease: sube +1 MSS por RTT, baja a la mitad ante pérdida.

CRÍTICA

- Tahoe **no distingue** entre timeout y 3 ACKs duplicados.
- Ante 3 ACKs duplicados la red **todavía está entregando** — no está totalmente colapsada.
- Aun así, Tahoe **reduce la VC a 1 MSS**.
- *Es demasiado conservador para ese caso; desperdicia capacidad.*



La crítica de Reno: ¿hace falta volver a VC = 1 MSS tras una pérdida detectada por ACKs duplicados?

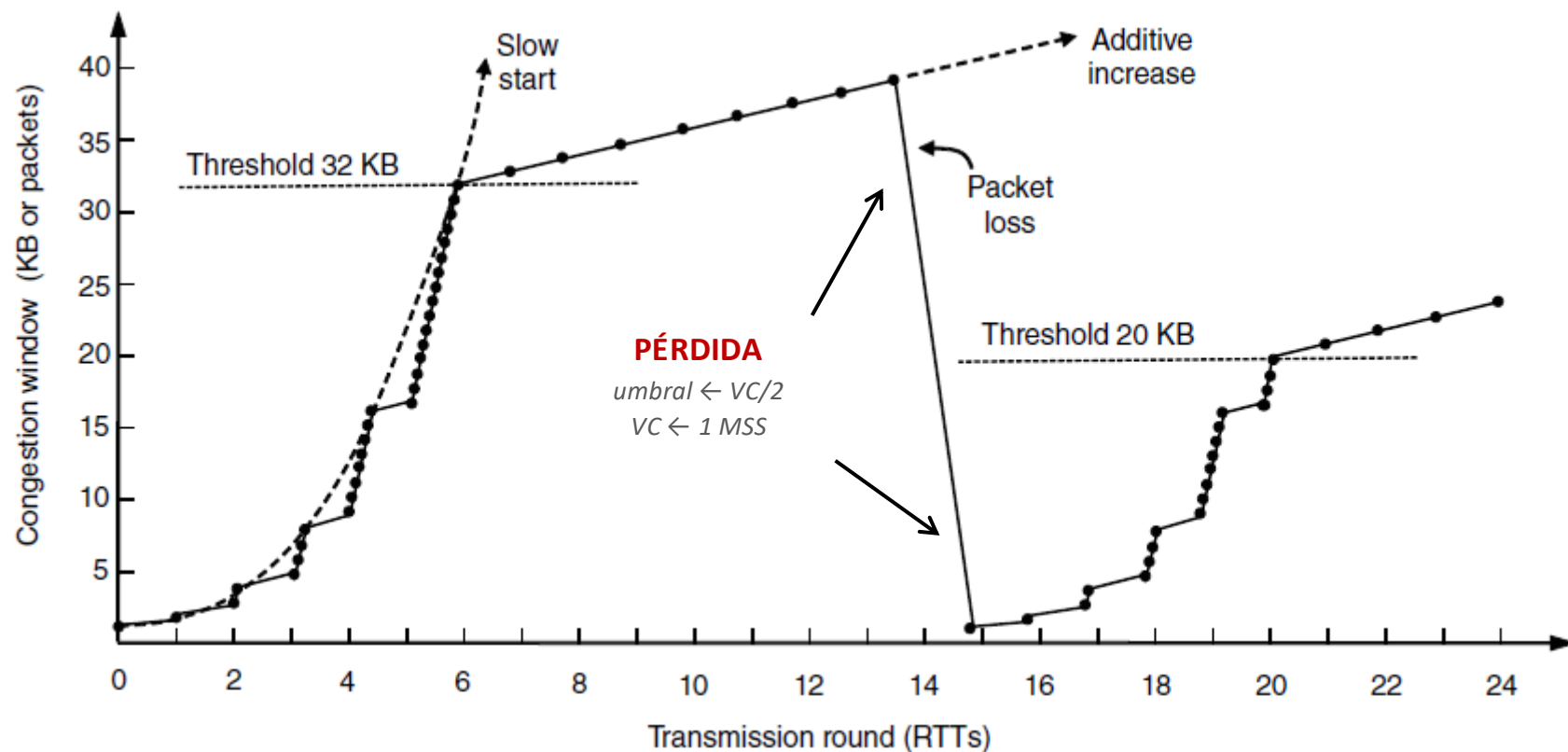
Resumen visual: TCP Tahoe en el tiempo

EVOLUCIÓN DE LA VC · LECTURA DEL DIAGRAMA

➤ **Exponencial** arranque lento — la VC dobla cada RTT hasta alcanzar el umbral.

➔ **Lineal** evitación de congestión — +1 MSS por RTT mientras no haya pérdidas.

↓ **Caída brusca** ante pérdida: umbral = $VC/2$ y VC vuelve a 1 MSS.



TCP Reno: mejorando a Tahoe

ALGORITMO DE CONTROL DE CONGESTIÓN (1990)

Reno trata distinto una pérdida detectada por ACKs duplicados que una por timeout

LA IDEA CLAVE

TIMEOUT

La red se colapsó

INTERPRETACIÓN

El "reloj de ACKs" se detuvo — probablemente ningún paquete está llegando.

ACCIÓN

Mismo tratamiento que Tahoe: $VC \leftarrow 1 MSS$, arranque lento.

3 ACKs DUPLICADOS

La red sigue andando

INTERPRETACIÓN

Están llegando paquetes posteriores: la red entrega, hay congestión pero no colapso.

ACCIÓN

Salta el arranque lento y entra en recuperación rápida.

Tahoe y Reno son iguales al inicio de la conexión y ante timeouts. Difieren solo ante 3 ACKs duplicados.

Recuperación rápida (**Fast Recovery**)

QUÉ HACE RENO TRAS 3 ACKS DUPLICADOS

La red sigue entregando paquetes — no vale la pena volver a arrancar desde 1 MSS

1 Retransmitir el paquete perdido (*retransmisión rápida*)

Los 3 ACKs duplicados repiten el mismo número de secuencia → ese es el segmento que falta. El emisor reenvía ese mismo segmento, sin esperar el timeout.

2 Ajustar umbral y ventana *$ssthresh \leftarrow VC/2$, $VC \leftarrow ssthresh$*

Se corta la VC a la mitad — pero sigue ahí, no vuelve a 1 MSS.

3 Enviar 1 segmento por cada ACK duplicado *$VC \leftarrow VC + 1$ por duplicado*

Mantiene paquetes en vuelo y "gasta" los ACK duplicados que aún llegan.

4 Al llegar un ACK nuevo, salir de fast recovery *se reanuda evitación de congestión*

El receptor confirmó que recibió el paquete perdido. VC vuelve a ssthresh y crece lineal.

El resultado: ciclos más chicos (VC cae a la mitad, no a 1 MSS) → menos desperdicio de ancho de banda.

Tahoe vs Reno: ¿qué cambia exactamente?

COMPARATIVA LADO A LADO

Evento	TCP Tahoe	TCP Reno
Inicio de conexión	Arranque lento	Arranque lento (igual)
Timeout	$VC \leftarrow 1 \text{ MSS}$, arranque lento	$VC \leftarrow 1 \text{ MSS}$, arranque lento (igual)
3 ACKs duplicados	$VC \leftarrow 1 \text{ MSS}$, arranque lento	$VC \leftarrow \text{ssthresh}$, recuperación rápida
Después de la pérdida	Vuelve a empezar desde 1	Sigue enviando con la mitad de la VC

La diferencia aparece solo ante 3 ACKs duplicados — en el resto de los casos ambos algoritmos se comportan igual.

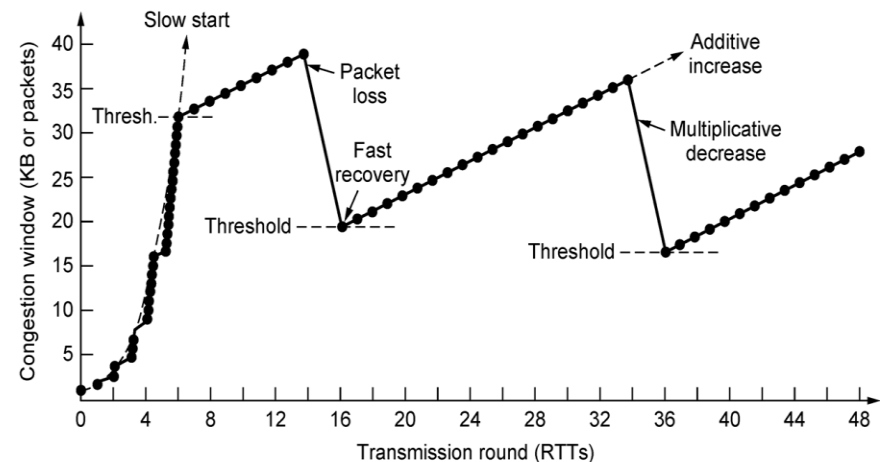
TCP Reno en el tiempo

VISIÓN CONJUNTA DE TODO EL CICLO

Menos caídas bruscas — mejor uso de la capacidad de la red

QUÉ MIRAR EN EL GRÁFICO

- Arranque lento solo al inicio **o tras timeout**.
- Ante 3 ACKs duplicados: **VC cae a la mitad**, no a 1 MSS.
- Luego sigue creciendo linealmente desde ssthresh.
- Los "dientes" son **más pequeños y frecuentes** que en Tahoe.



Fast recovery and the sawtooth pattern of TCP Reno.

Reno sigue siendo "diente de sierra", pero con dientes más chicos. Es la base de los TCP actuales (NewReno, Cubic...).

¿La congestión solo se detecta por pérdidas?

No: también por delay (RTT)

Hasta ahora: 📌 “si hay pérdida → hay congestión”

Problema de usar solo pérdidas.

Antes de que haya pérdidas:

- Las colas en routers crecen
- El RTT aumenta
- Se acumula delay

Señales basadas en delay

Idea: 📌 usar el aumento del RTT como indicador temprano

- RTT esperado (sin cola)
- RTT medido (con cola)

Si 📌 RTT ↑ ⇒ hay cola ⇒ hay congestión

 Ejemplo 1 - TCP **Vegas**: Compara Throughput esperado (sin congestión) y real

 Ejemplo 2 - **BBR** (Google): Modelo basado en Bandwidth y RTT mínimo

La congestión aparece antes del drop. El delay permite detectarla antes

Justicia (*Fairness*) en TCP

¿Cómo se reparte la red entre múltiples flujos?

Cuando varios flujos comparten un mismo enlace: 🖐️ ¿quién usa cuánto?

⚖️ Un sistema es fair si: Flujos similares obtienen igual throughput

Ejemplo:

- * 2 flujos → ~50% cada uno
- * 3 flujos → ~33% cada uno

⚠️ Pero... no siempre es justo

Problemas de fairness:

- Flujos con menor RTT → ganan más ancho de banda
- TCP vs UDP → UDP puede ser “injusto”
- TCP Reno vs Vegas → los delay-based pierden
- Flujos cortos vs largos → los cortos pierden

📈 Cómo TCP logra fairness (AIMD)

TCP usa:

- Additive Increase (sube lento)
- Multiplicative Decrease (baja fuerte)

🖐️ Resultado:

- Flujos agresivos → pierden más → bajan
- Flujos lentos → crecen progresivamente

➡️ Convergencia hacia equilibrio

AIMD ≈ mecanismo simple que produce reparto relativamente justo, pero la fairness real depende de RTT, algoritmos y competencia

Control de flujo + Control de congestión en TCP

Cómo se combinan en la práctica

Control de flujo (receptor)
rwnd (receive window)

Control de congestión (red)
cwnd (congestion window)

Ventana efectiva de envío
 $\min(\text{rwnd}, \text{cwnd})$

Insight importante

- cwnd es dinámico (AIMD, pérdidas, RTT)
- rwnd depende de la aplicación (lectura del buffer)

Ambas ventanas son independientes, pero se combinan en el envío

👉 *TCP envía datos al ritmo del más restrictivo entre la red y el receptor*

PARTE FINAL

Ejercicios de repaso

Tres problemas sobre control de congestión con TCP — con solución paso a paso

1. Concepto · 2. Cálculo · 3. Identificaciones

Ejercicio: comportamiento de TCP Tahoe

PARA PRACTICAR EN CLASE

PLANTEO

Se usa **TCP Tahoe**; la VC está en **36 KiB** y ocurre un **timeout**. Tras el timeout, la VC llega hasta **24 KB** sin más timeouts. Tamaño máximo de segmento: **1 KB**.

RESPONDER

1 ¿Qué representan los ejes del diagrama?

Definí qué magnitud va en cada uno — eje horizontal (X) y eje vertical (Y).

2 Dibujar el comportamiento de la VC

Desde el timeout (VC = 36 KB) hasta que la VC alcanza 24 KB sin nuevos timeouts.

UMBRAL TRAS TIMEOUT
ssthresh = 18 KB

VC DE PARTIDA
1 KB

CAMBIO DE FASE
a los 18 KB

Sugerencia: recordar que $ssthresh \leftarrow VC/2$ y $VC \leftarrow 1 \text{ MSS}$ al timeout; luego exponencial hasta ssthresh y después lineal.

Ejercicio conceptual: ¿congestión o control de flujo?

EJERCICIO CONCEPTUAL · DISTINGUIR DOS PROBLEMAS

PLANTEO

Identificar en cada caso si el problema es CONGESTIÓN (red saturada) o CONTROL DE FLUJO (receptor lento).

CASO A

Un servidor envía video a un cliente. El cliente tiene buffer grande pero los routers intermedios empiezan a descartar paquetes.

→ **RESPUESTA** Congestión

El receptor podría procesar más, pero la red no lo transporta — problema dentro de la subred.

CASO B

Un servidor envía a alta velocidad a un sensor IoT de poca memoria. El sensor anuncia ventana cada vez más chica.

→ **RESPUESTA** Control de flujo

La red tiene capacidad; el que no aguanta el ritmo es el receptor. Se resuelve con la ventana anunciada.

CASO C

El cliente recibe ACKs llegando tarde y algunos segmentos aparecen duplicados. El RTT observado se disparó.

→ **RESPUESTA** Congestión

RTTs crecientes y pérdidas = colas llenándose en routers. TCP reacciona bajando la VC.

CASO D

Dos usuarios en la misma WiFi descargan archivos grandes y la velocidad total se reparte bien — sin pérdidas.

→ **RESPUESTA** Ninguno

Sin pérdidas ni ventana de receptor reducida: TCP y la red están balanceando correctamente.

Regla de oro: congestión = problema DENTRO de la red · control de flujo = problema EN EL RECEPTOR.

Ejercicio de cálculo: evolución de VC tras un timeout

EJERCICIO DE CÁLCULO · TCP TAHOE

PLANTEO

En TCP Tahoe, la VC estaba en **16 MSS** cuando se produce un timeout. ¿Cuál es el valor de la VC durante los próximos 5 RTTs?

RESOLUCIÓN · EVOLUCIÓN DE LA VC POR RTT

RTT	VC (MSS)	FASE	
0	16 → 1	—	Timeout: $ssthresh \leftarrow 16/2 = 8$; $VC \leftarrow 1$ MSS.
1	2	exponencial	Arranque lento: VC duplica ($1 \rightarrow 2$).
2	4	exponencial	Sigue duplicando ($2 \rightarrow 4$).
3	8	exponencial	$4 \rightarrow 8$, alcanza el umbral ($ssthresh = 8$).
4	9	lineal	Evitación de congestión: +1 MSS por RTT.
5	10	lineal	$9 \rightarrow 10$. Sigue el crecimiento lineal.

Respuesta: VC = {1, 2, 4, 8, 9, 10} en los RTTs {0...5} — 3 RTTs de fase exponencial + 2 RTTs lineales.

Tahoe vs Reno frente a 3 ACKs duplicados

EJERCICIO CONCEPTUAL · COMPARAR COMPORTAMIENTOS

PLANTEO

Un emisor con **VC = 20 MSS** recibe **3 ACKs duplicados**. ¿Cómo reacciona cada algoritmo?

EN TAHOE

- ▶ $\text{ssthresh} \leftarrow \text{VC}/2 = \mathbf{10 \text{ MSS}}$
- ▶ $\text{VC} \leftarrow \mathbf{1 \text{ MSS}}$
- ▶ Retransmite el paquete perdido
- ▶ Vuelve a **arranque lento**
- ▶ Tarda varios RTTs en recuperar

EN RENO

- ▶ $\text{ssthresh} \leftarrow \text{VC}/2 = \mathbf{10 \text{ MSS}}$
- ▶ $\text{VC} \leftarrow \text{ssthresh} = \mathbf{10 \text{ MSS}}$
- ▶ Retransmite el paquete perdido
- ▶ Entra en **recuperación rápida**
- ▶ Sigue enviando sin reiniciar

Insight: Reno asume que si siguen llegando ACKs (aunque duplicados), la red sigue andando — no vale la pena reiniciar.

¿Cuántos RTTs tarda en transferir el archivo?

EJERCICIO DE CÁLCULO · ARRANQUE LENTO PURO

PLANTEO

Se quiere transferir un archivo de **64 KB**. El tamaño máximo de segmento es **4 KB**. No hay pérdidas. ¿En qué RTT se completa la transferencia asumiendo arranque lento puro?

RESOLUCIÓN · BYTES ENVIADOS POR RTT

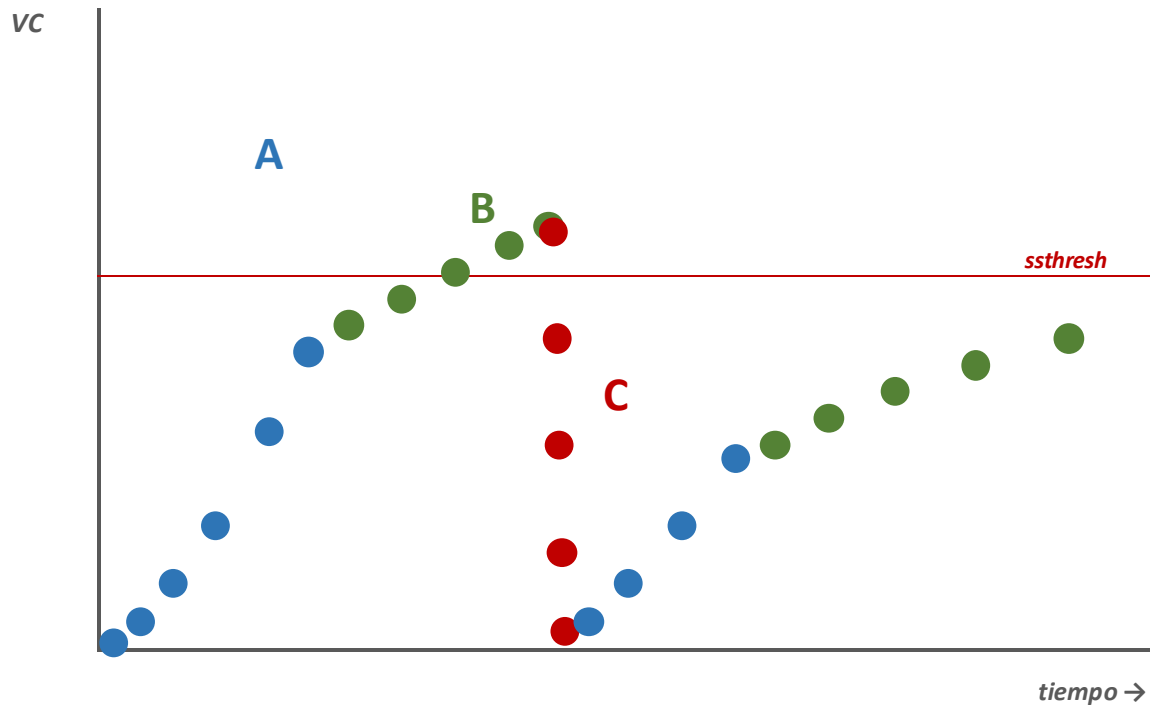
RTT	VC	ENVIADO	ACUMULADO — ESTADO
1	1 seg	4 KB	4 KB — <i>arranque, 60 KB faltan</i>
2	2 seg	8 KB	12 KB — <i>52 KB faltan</i>
3	4 seg	16 KB	28 KB — <i>36 KB faltan</i>
4	8 seg	32 KB	60 KB — <i>4 KB faltan</i>
5	16 seg (cap)	4 KB	64 KB — <i>¡completado!</i>

Respuesta: en el RTT 5 se completa la transferencia (total acumulado = 64 KB).

Identificar las fases en un gráfico de VC

EJERCICIO INTEGRADOR · ANALIZAR EL COMPORTAMIENTO

Observá el gráfico y etiquetá cada segmento con la fase correspondiente



IDENTIFICÁ CADA FASE

A
(anotá aquí)

B
(anotá aquí)

C
(anotá aquí)

Tip: fijate si la VC crece exponencialmente, linealmente, o cae de golpe.